



Information Technology
Generalist

PROG1400 – Midterm Practical Instructions

Course: PROG1400 – Introduction to Object-Oriented Programming

Instructor: Davis Boudreau

Evaluation Type: Midterm Practical (Hands-On) **Duration:** 2 Hours (In-Class Lab)

Purpose of This Midterm

This midterm evaluates your ability to **apply object-oriented programming concepts in practice**, not just describe them.

You will demonstrate that you can:

- Model a system using **UML class and sequence diagrams**
- Use **Mermaid diagrams inside VS Code**
- Use **Copilot responsibly**
- Apply **object-oriented design principles to your game**
- Manage your work using **GitHub version control**
- Test your design by **instantiating objects**
- Explain your design decisions

This midterm is based on **your Pac-Man-like game project** selected earlier in the course.

Learning Outcomes Being Assessed

This midterm assesses the following learning outcomes:

LO2 – Implement object-oriented design principles in applications

LO3 – Manage and distribute code through code reuse and versioning

LO4 – Develop an object-oriented solution using software modelling documentation

Allowed Tools

You **may use**:

- VS Code
- GitHub repository
- Mermaid diagrams
- GitHub Copilot
- Course notes

You **may NOT**:

- Copy work from another student
- Accept Copilot output without understanding it

You must be able to **explain everything you submit**.

Midterm Development Workflow

During the exam you should follow the **professional development pipeline** handed out by your instructor.

Task 1 – Repository Setup & Version Control

Learning Outcome: **L03**

You will begin by setting up a **GitHub repository** that will contain your midterm work.

Step 1 – Create GitHub Repository

Create a **public repository** named:

```
wXXXXXXX-midterm
```

Where:

```
XXXXXXX = your NSCC student number
```

Example:

```
w1234567-midterm
```

Step 2 – Clone the Repository

Clone the repository to your computer.

Example:

```
git clone <repository-url>
```

Open the repository in **VS Code**.

Step 3 – Create Project Structure

Inside the repository create:

```
docs/  
  uml/  
src/
```

Your project should resemble:

```
wXXXXXXX-midterm  
├── docs  
│   └── uml  
│       ├── initialization_state_class_diagram.mmd  
│       └── initialization_state_sequence_diagram.mmd  
├── src  
│   └── game_objects.py  
└── README.md
```

Step 4 – Commit Incrementally

Make **small commits throughout the midterm**.

Example commit messages:

```
Initial repository setup  
Added UML class diagram  
Added UML sequence diagram  
Implemented game object classes  
Added instantiation testing
```

A single commit at the end does not demonstrate proper version control.

Task 2 – UML Modelling with Mermaid

Learning Outcome: **LO4**

You will model the **Initialization State of your game**.

Step 1 – Identify Game Objects

Identify **3–5 key objects** that exist when your game **initializes**.

Examples may include:

```
GameController
Player
Enemy
Maze
Pellet
ScoreManager
```

Step 2 – Create UML Class Diagram

Create the file:

```
docs/uml/initialization_state_class_diagram.mmd
```

Your diagram must include:

- 3–5 classes
- attributes
- methods
- relationships between classes

Example structure:

```
GameController --> Player
GameController --> Enemy
Player --> ScoreManager
```

Step 3 – Create UML Sequence Diagram

Create:

```
docs/uml/initialization_state_sequence_diagram.mmd
```

This diagram should show **how objects interact during game startup**, for example:

- Game starts
- Player object created
- Maze loaded
- Enemies spawned
- Score system initialized

Example Mermaid sequence structure:

```
sequenceDiagram
GameController->>Player: create()
GameController->>Maze: load()
GameController->>Enemy: spawn()
GameController->>ScoreManager: initialize()
```

Copilot Use

You may use **GitHub Copilot** to assist with Mermaid syntax.

However, you must be able to explain:

- What each class represents
- Why relationships exist
- What the sequence diagram shows

Task 3 – Apply OOP Principles to Your Game

Learning Outcome: **L02**

Using your UML diagram as a guide, you will implement **Python classes that represent the objects in your game**.

Your code should demonstrate the following principles applied to **your game objects**:

- Encapsulation
- Inheritance
- Polymorphism

Step 1 – Create Python Source File

Create the file:

```
src/game_objects.py
```

Step 2 – Implement Classes from Your Game Design

Your classes should reflect the objects identified in your UML diagram.

For example:

```
Player  
Enemy  
GameController  
Maze  
ScoreManager
```

Your code should demonstrate **clear object responsibilities**.

Step 3 – Test by Instantiating Objects

Your program must demonstrate that objects can be created and interact.

Example structure:

```
game = GameController()  
  
player = Player()  
enemy = Enemy()  
  
player.update()  
enemy.update()
```

This confirms that:

- Classes exist
 - Objects can be instantiated
 - Methods execute
-

OOP Memory Jogger (Non-Game Examples)

The following examples illustrate **core OOP principles using non-game contexts**.

These are **reference examples only**.

Encapsulation Example (Bank Account)

```
class BankAccount:

    def __init__(self):
        self._balance = 1000

    def deposit(self, amount):
        self._balance += amount

    def get_balance(self):
        return self._balance
```

The account object manages its own financial data.

Inheritance Example (Vehicle System)

```
class Vehicle:

    def start(self):
        print("Vehicle starting")

class Car(Vehicle):
    pass
```

The Car class inherits behavior from Vehicle.

Polymorphism Example (Library System)

```
class LibraryItem:

    def get_description(self):
        pass

class Book(LibraryItem):

    def get_description(self):
        print("This is a book")

class DVD(LibraryItem):

    def get_description(self):
        print("This is a DVD")
```

Different objects respond differently to the same method.

Expected Final Repository

Your repository should contain:

```
wXXXXXXXX-midterm
├── docs
│   └── uml
│       ├── initialization_state_class_diagram.mmd
│       └── initialization_state_sequence_diagram.mmd
├── src
│   └── game_objects.py
└── README.md
```

How Tasks Align With Learning Outcomes

Task	Learning Outcome
Repository creation & commits	LO3 – Version control
UML modelling	LO4 – Software modelling documentation
Python implementation	LO2 – OOP design principles

What Will Be Evaluated

You will be evaluated on:

- Time management
- Problem solving
- Use of information resources
- Understanding of UML and OOP
- Hands-on technical skills

Tools evaluated include:

- VS Code
- GitHub
- Mermaid
- Python

Tips for Success

✔ Think before coding ✔ Commit frequently ✔ Keep UML diagrams simple and accurate ✔ Ensure your code reflects your UML design ✔ Be ready to explain your work

Common Mistakes to Avoid

✗ Waiting until the end to commit ✗ UML diagrams that do not match your code ✗ One class doing everything ✗ Accepting Copilot output without understanding it ✗ Not testing your objects

Submission

Your **GitHub repository is your submission.**

Ensure it contains:

- UML diagrams
 - Python source files
 - Commit history showing your workflow
-

Midterm Practical Rubric

Total Score: 25

Criteria	Points
Time Management	/5
Problem Solving Skills	/5
Information Resource Use	/5
Knowledge of UML & OOP	/5
Hands-on Technical Skills	/5

Total Midterm Score

/25
